The Command Pattern

Turning requests into objects

A short conceptual introduction

Design Patterns Workshop

Learning Goal

By the end of this introduction, you should be able to explain:

- What problem the Command pattern solves
- What a Command actually represents
- The roles: Client, Command, Invoker, Receiver
- Why this reduces coupling
- When the pattern becomes useful

The Problem

Imagine a text editor with many ways to trigger the same operation.

Toolbar

Copy / Cut / Paste

Context menu

Copy / Cut / Paste

Keyboard

Ctrl+C / Ctrl+X / Ctrl+V

Where should the actual operation live?

The Naive Solution

Let each UI element know how to perform the operation.

Button

↓

Editor.copy()

MenuItem

↓

Editor.copy()

Shortcut

↓

Editor.copy()

Same operation — duplicated knowledge.

The Key Idea

Instead of passing a request directly, turn it into an object.

The request itself becomes a first-class object.

Commands

A Command turns a request into a stand-alone object.

A request

"Delete these files"

"Send this message"

"Save this document"

A Command object

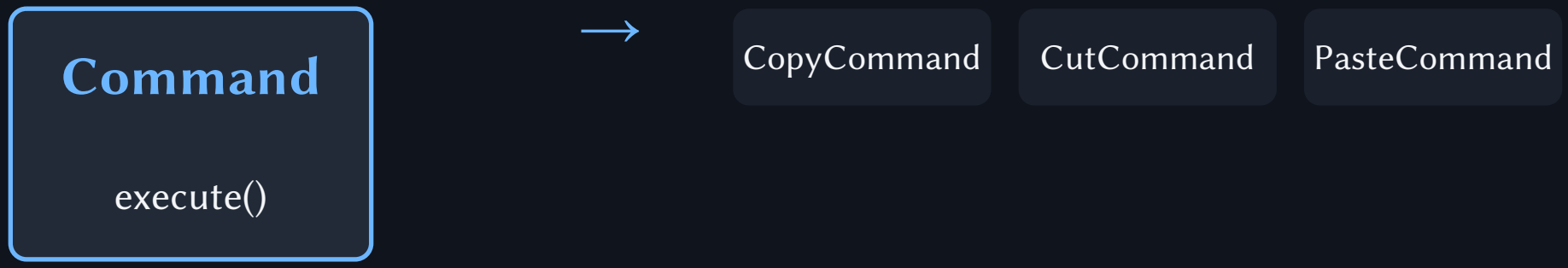
Contains the information needed to execute the request.

Typically:

- the receiver
- the operation
- the required parameters

Common Interface

The caller doesn’t need to know which concrete operation it is triggering.



Different operations · same interface

The Main Roles

Client

Creates and configures commands.

Invoker

Triggers a command. Does not perform the business operation.

Command

Defines the common execution interface.

Receiver

Contains the actual business logic.

The Interaction

Each role has a different responsibility.

The Invoker doesn’t need to know how the operation works.

The Decoupling

Before

UI element

↓

concrete business logic

The caller needs to understand the operation.

After

UI element

↓

Command

↓

business logic

The caller only depends on the Command interface.

The Consequence

A command can be treated like data.

- Pass it around
- Store it
- Replace it
- Queue it
- Delay its execution
- Log or serialize it

Execution becomes something we can manage.

Undo & Redo

Commands can represent a history of operations.

Command history

← undo

A command may contain enough information to restore the previous state or perform an inverse operation.

The Usage

- You want to parameterize an object with an operation.
- Several different UI elements should trigger the same operation.
- Operations need to be queued or scheduled.
- Operations should be logged, stored, or sent elsewhere.
- You need undo / redo or another form of operation history.

The Price

More indirection.

- A simple method call may become several objects.
- There are more classes and interfaces to understand.
- The pattern can be unnecessary for very simple code.

The Mental Model

Don’t ask:

“How should the button perform this operation?”

Ask:

“What object represents this operation?”

Cheat Sheet

Client

Creates the command

→

Command

Represents the operation

→

Receiver

Performs the operation

Invoker

execute()

“I don’t need to know what the command does.”

Now let’s build one.

We will start with the problem, then introduce the pattern step by step.

Focus question:

“What should know what?”

Source

Based on the Command pattern overview by Refactoring.Guru.

Command is described as a behavioral design pattern that turns a request into a stand-alone object containing the information required for that request.

Reference:

Refactoring.Guru — Design Patterns / Command